

Bayesian Hierarchical MPT Modeling

Application and Practice with TreeBUGS

Daniel W. Heck



2025-03-13

TreeBUGS: Bayesian Hierarchical MPT Modeling

- 1) The R package TreeBUGS
- 2) Basic modeling
 - Model fitting
 - Convergence
 - Plots
 - Model fit
- 3) Advanced modeling
 - Within-subject comparisons
 - Between-subject comparisons
 - Covariates
- 4) Sensitivity/robustness analysis
 - Priors
 - Predictive distributions
 - Simulation

Software for Hierarchical MPTs

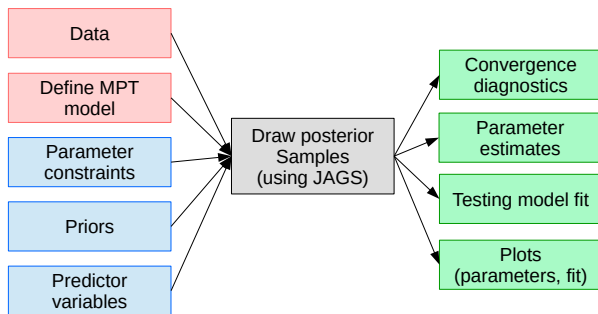
- Implementation of MCMC sampling in R/C/Fortran (Klauer, 2010)
- General-purpose software: WinBUGS/JAGS/Stan (Matzke et al., 2015)
- Requires re-implementation of summaries, statistics, plots

TreeBUGS: A user-friendly R package (Heck et al., 2018)

- Easy-to-use, open source, free
- Fitting and testing MPT models
 - Posterior sampling, summary statistics, and plots
 - Data generation, robustness simulations
 - Change priors, add predictors, etc.
 - Current limitation: Crossed random effects (persons + items)

Functionality of TreeBUGS

- Input: R objects or text/csv files (minimal R knowledge required)
- Priors and other details can be changed in R
- TreeBUGS translates the model to JAGS (Plummer, 2003) to draw posterior samples
- Functions for post-processing, summaries and plots



TreeBUGS Paper

Behav Res

DOI 10.3758/s13428-017-0869-7



TreeBUGS: An R package for hierarchical multinomial-processing-tree modeling

Daniel W. Heck¹ · Nina R. Arnold¹ · Denis Arnold^{2,3}

© The Author(s) 2017. This article is published with open access at Springerlink.com

Abstract Multinomial processing tree (MPT) models are a class of measurement models that account for categorical data by assuming a finite number of underlying cognitive processes. Traditionally, data are aggregated across participants and

estimates, fit statistics, and within- and between-subjects comparisons, as well as goodness-of-fit and summary plots. We also propose and implement novel statistical extensions to include continuous and discrete predictors (as either fixed or

Basic Modeling

(corresponding R script: 06-ApplicationIII-TreeBUGS.R)

MPT Model Specification

Model equations

- MPT structure is defined in an EQN model file
- Simple: copy from multiTree :-)
- Difference: the symbol # allows to add comments

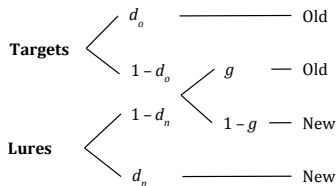
Example: Two-high threshold model (file: 2htm.eqn)

Targets

```
target hit do
target hit (1-do)*g
target miss (1-do)* (1-g)
```

Lures

```
lure cr dn
lure fa (1-dn)*g
lure cr (1-dn)*(1-g)
```



Data matrix

- Response frequencies in wide format
- either .csv-file or data.frame / matrix in R
- One line per person
- One category per column
- Column names must be identical to the EQN categories!

Example: 2htm.csv

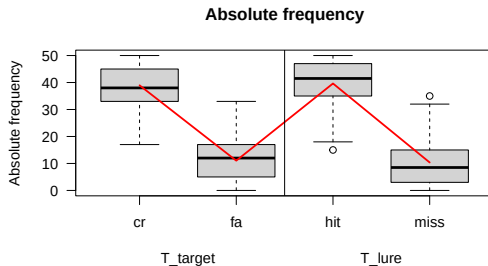
```
# set working directory:  
# setwd("D:/R/MPT-workshop/")  
frequencies <- read.csv("2htm.csv")  
head(frequencies, 5)
```

```
##   cr fa hit miss  
## 1 43  7  49    1  
## 2 45  5  46    4  
## 3 26 24  29   21  
## 4 34 16  22   28  
## 5 41  9  26   24
```

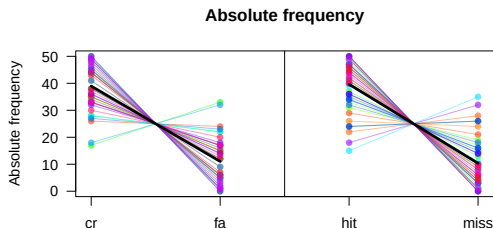
Heterogeneity

■ Load TreeBUGS and plot heterogeneity

```
library(TreeBUGS)
plotFreq(frequencies, eqnfile = "2htm.eqn")
```



```
plotFreq(frequencies, boxplot = FALSE, eqnfile = "2htm.eqn")
```



Fitting an MPT model in TreeBUGS

- Model: Text file in EQN syntax (with model equations)
- Data: .csv file
- Constraints: text file with equality constraints

```
fit <- traitMPT(eqnfile = "htm.txt",  
               data = "responses.csv",  
               restrictions = "2htm_constraints.txt")  
  
# beta-MPT: different function, but identical arguments  
fit_beta <- betaMPT(eqnfile = "htm.txt",  
                   data = "responses.csv",  
                   restrictions = "2htm_constraints.txt")
```

Fitting an MPT Model in R

Alternative approach: define everything directly in R

- Model: Text string (character in apostrophes)
- Data: Matrix or data frame
- Constraints: a list

```
htm <- "  
target hit do  
target hit (1-do)*g  
target miss (1-do)* (1-g)  
  
lure cr dn  
lure fa (1-dn)*g  
lure cr (1-dn)*(1-g)  
"  
fit <- traitMPT(eqnfile = htm,  
               data = frequencies,  
               restrictions = list("dn=do", "g=.50"))
```

Parameter Constraints

Equality constraints

(A) use a general model file and constrain parameters:

```
fit <- traitMPT(eqnfile = htm,  
               data = frequencies,  
               restrictions = list("dn=do", "g=.50"))
```

(B) hard-coding of constraints in the EQN file:

```
htm_constr <- "  
target hit d  
target hit (1-d)*.50  
target miss (1-d)*.50  
  
lure cr d  
lure fa (1-d)*.50  
lure cr (1-d)*.50  
"  
fit <- traitMPT(eqnfile = htm_constr,  
               data = frequencies)
```

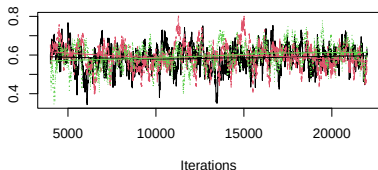
Convergence

How to check the convergence?

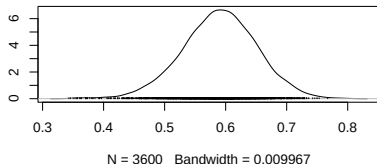
- Posterior/ MCMC samples should look unsystematic (like a hairy caterpillar)
- For more options, see: `?plot.traitMPT`

```
plot(fit, parameter = "mean", type = "default")
```

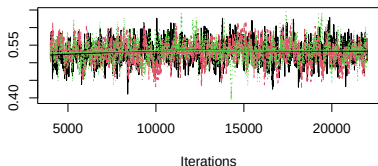
Trace of mean[dn]



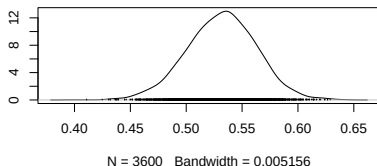
Density of mean[dn]



Trace of mean[g]



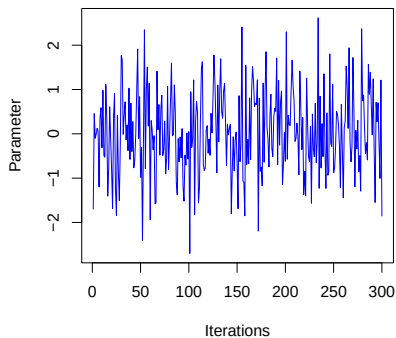
Density of mean[g]



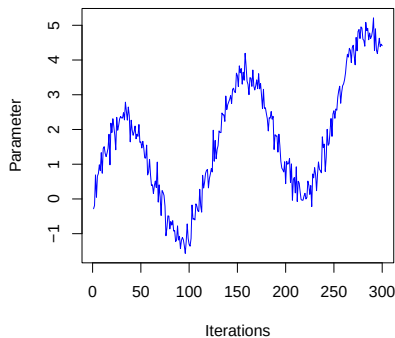
Convergence

■ Interpreting MCMC plots

Good convergence: 'hairy caterpillar'



Bad convergence: slow movement

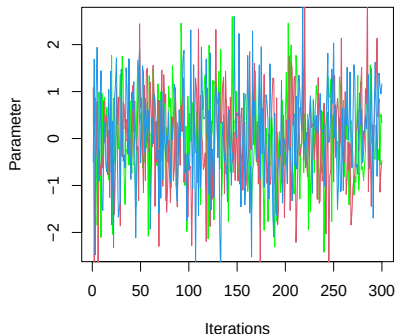


Convergence

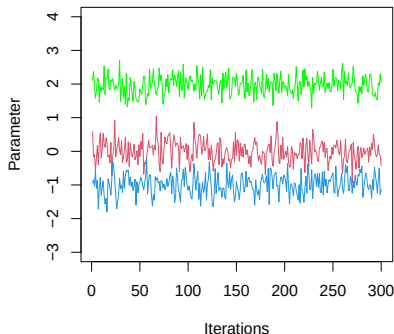
Gelman-Rubin statistic

- Also known as: “potential scale reduction factor” (psrf) or “R hat” ($\hat{R}$)
- Similar logic as ANOVA: Comparison of between-chain and within-chain variances (a large difference between these variances indicates nonconvergence)
- The statistic should be close to 1 (standard criterion: $\hat{R} < 1.05$)

Good convergence: all chains similar



Bad convergence: chains differ



Convergence

- Gelman-Rubin statistic ($\hat{R}$): Columns Rhat and R_95% in the summary output

```
summary(fit)
```

```
## Call:
## traitMPT(eqnfile = htm, data = frequencies, restrictions = list("dn=do"),
##         ppp = 1000)
##
## Group-level medians of MPT parameters (probability scale):
##           Mean    SD  2.5%   50%  97.5% Time-series SE n.eff  Rhat R_95%
## mean_dn  0.590 0.062 0.465 0.591 0.708         0.004   268 1.021 1.073
## mean_g   0.532 0.031 0.468 0.532 0.593         0.001   782 1.012 1.038
##
## Mean/Median of latent-trait values (probit-scale) across individuals:
##           Mean    SD  2.5%   50%  97.5% Time-series SE n.eff  Rhat R_95%
## latent_mu_dn 0.23 0.161 -0.089 0.231 0.547         0.010   267 1.022 1.074
## latent_mu_g  0.08 0.079 -0.080 0.082 0.234         0.003   782 1.012 1.038
##
## Standard deviation of latent-trait values (probit scale) across individuals:
##           Mean    SD  2.5%   50%  97.5% Time-series SE n.eff  Rhat R_95%
## latent_sigma_dn 1.088 0.143 0.844 1.075 1.399         0.003  2688 1.003 1.009
## latent_sigma_g  0.409 0.070 0.287 0.404 0.559         0.001  3015 1.002 1.007
##
## Correlations of latent-trait values on probit scale:
##           Mean    SD  2.5%   50%  97.5% Time-series SE n.eff  Rhat R_95%
## rho[dn,g] 0.142 0.199 -0.267 0.148 0.509         0.005  1860    1 1.002
##
## Correlations (posterior mean estimates) in matrix form:
##           dn      g
```

Options for MCMC Sampling

- If the model has not converged, it must be fitted with more conservative settings:

```
fit <- traitMPT(  
  eqnfile = htm, data = frequencies,  
  restrictions = list("dn=do"),  
  
  n.adapt = 5000, # longer adaption of JAGS increases efficiency of sampling  
  n.burnin = 5000, # longer burnin avoids issues due to bad starting values  
  n.iter = 30000, # drawing more MCMC samples leads to higher precision  
  n.thin = 10, # omitting every 10th sample reduces memory load  
  n.chains = 4) # more MCMC chains increase precision
```

```
## MCMC sampling started at 2025-03-12 15:22:19  
## Calling 4 simulations using the parallel method...  
## Following the progress of chain 1 (the program will wait for all chains  
## to finish before continuing):  
## Welcome to JAGS 4.3.1 on Wed Mar 12 15:22:23 2025  
## JAGS is free software and comes with ABSOLUTELY NO WARRANTY  
## Loading module: basemod: ok  
## Loading module: bugs: ok  
## . Loading module: dic: ok  
## . Loading module: glm: ok  
## . . Reading data file data.txt  
## . Compiling data graph  
## Resolving undeclared variables  
## Allocating nodes
```

Extend MCMC Sampling

- Sometimes, MCMC samples are OK but higher precision is needed
- Extend sampling and add new MCMC samples to the fitted JAGS object:

```
fit2 <- extendMPT(fit,           # fitted MPT model
                  n.adapt = 2000, # JAGS need to restart and adapt again
                  n.burnin = 0,   # burnin not needed if previous samples are OK
                  n.iter = 10000) # how many additional iterations?
```

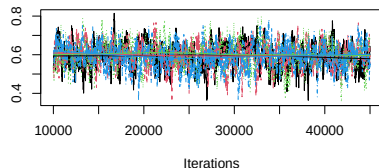
```
## Calling 4 simulations using the parallel method...
## Following the progress of chain 1 (the program will wait for all chains
## to finish before continuing):
## Welcome to JAGS 4.3.1 on Wed Mar 12 15:22:46 2025
## JAGS is free software and comes with ABSOLUTELY NO WARRANTY
## Loading module: basemod: ok
## Loading module: bugs: ok
## . Loading module: dic: ok
## . Loading module: glm: ok
## . . Reading data file data.txt
## . Compiling data graph
##   Resolving undeclared variables
##   Allocating nodes
##   Initializing
##   Reading data back into data table
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 100
##   Unobserved stochastic nodes: 55
##   Total graph size: 1094
```

Convergence for Second Fit

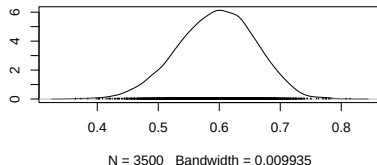
- Check convergence again:

```
plot(fit2, parameter = "mean", type = "default")
```

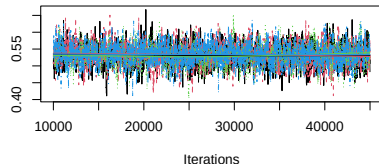
Trace of mean[dn]



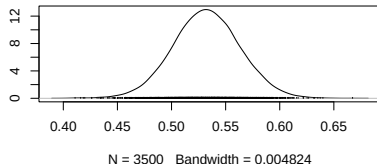
Density of mean[dn]



Trace of mean[g]



Density of mean[g]



Summary statistics for posterior distribution

- Posterior mean and median (50% quantile)
- Posterior standard deviation (SD, similar to standard error)
- Bayesian credibility interval (2.5% and 97.5% quantiles)

```
summary(fit2)
```

```
## Call:
## [[1]]
## traitMPT(eqnfile = htm, data = frequencies, restrictions = list("dn=do"),
##   n.iter = 30000, n.adapt = 5000, n.burnin = 5000, n.thin = 10,
##   n.chains = 4)
##
## [[2]]
## extendMPT(fittedModel = fit, n.iter = 10000, n.adapt = 2000,
##   n.burnin = 0)
##
##
## Group-level medians of MPT parameters (probability scale):
##      Mean    SD  2.5%   50% 97.5% Time-series SE n.eff  Rhat R_95%
## mean_dn 0.595 0.063 0.467 0.597 0.711      0.003   633 1.004 1.007
## mean_g  0.532 0.031 0.471 0.532 0.594      0.001  2275 1.002 1.005
##
## Mean/Median of latent-trait values (probit-scale) across individuals:
##      Mean    SD  2.5%   50% 97.5% Time-series SE n.eff  Rhat R_95%
## latent_mu_dn 0.244 0.165 -0.084 0.247 0.557      0.006   652 1.004 1.007
## latent_mu_g  0.081 0.079 -0.073 0.081 0.237      0.002  2276 1.002 1.005
##
## Standard deviation of latent-trait values (probit scale) across individuals:
##      Mean    SD  2.5%   50% 97.5% Time-series SE n.eff  Rhat R_95%
## latent_sigma_dn 1.089 0.144 0.846 1.074 1.409      0.002  4230 1.004 1.012
```

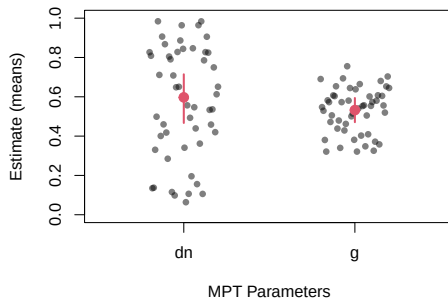
Plot Parameter Estimates

Estimates for group-level parameters

- Overall mean μ : Posterior mean and Bayesian credibility interval
- Individual parameters θ_i : Posterior mean

```
plotParam(fit)
```

roup-level means + 95% CI (red) and individual means



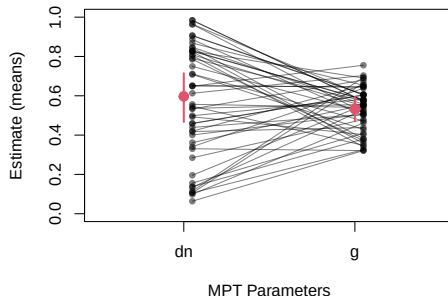
Plot Parameter Estimates

Illustration of parameter correlations

- Plot a profile of individual parameters
- e.g., for assessing test-retest reliability of a parameter (Michalkiewicz and Erdfelder, 2016)
- For more options, see: `?plotParam`

```
plotParam(fit, addLines = TRUE, select = c("dn", "g"))
```

roup-level means + 95% CI (red) and individual means



Compare Prior and Posterior

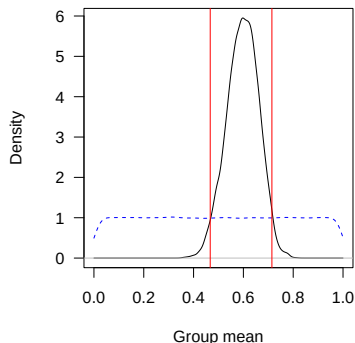
How much did we learn about the parameters?

- Graphical assessment: Plot prior (blue) and posterior (black) densities

```
plotPriorPost(fit)
```

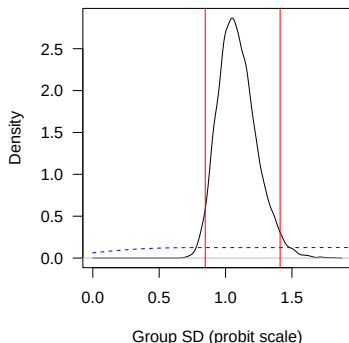
```
## Press <Enter> to show the next plot.
```

Group mean of dn



```
## Press <Enter> to show the next plot.
```

Group SD of dn



Group mean of g

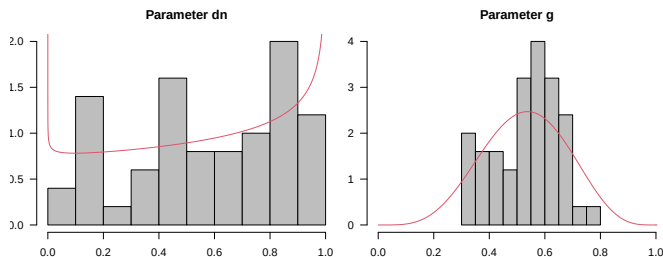
Group SD of g

Group-Level Distribution

Distribution of individual estimates

- Histogram: Distribution of θ estimates (posterior mean per person)
- Red density: Estimated group-level distribution

```
plotDistribution(fit) # graphical test
```



Model Fit: Predicted vs. Observed Data

Graphical test of model fit: Means

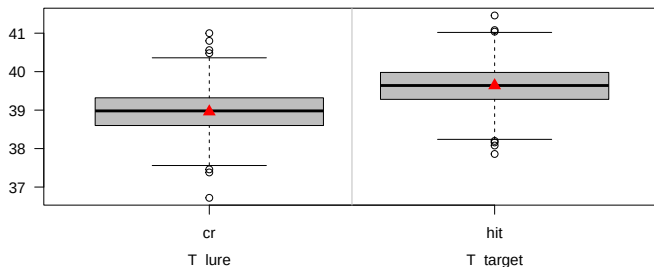
- Plot the means of the observed frequencies
- Comparison against the posterior-predicted frequencies (boxplots)

```
colMeans(frequencies)  # observed group means that are tested
```

```
##      cr      fa    hit  miss  
## 38.96 11.04 39.64 10.36
```

```
plotFit(fit)           # graphical test
```

Observed (red) and predicted (boxplot) mean frequencies



Model Fit: Predicted vs. Observed Data

Graphical test of model fit: Covariances

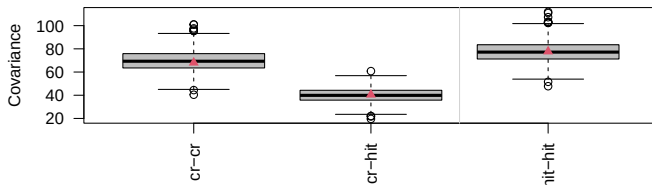
- Plot the *covariances* of the observed/posterior-predicted frequencies

```
cov(frequencies)           # observed covariance matrix that is tested
```

```
##           cr          fa          hit          miss
## cr      68.08000 -68.08000  40.49551 -40.49551
## fa     -68.08000  68.08000 -40.49551  40.49551
## hit    40.49551 -40.49551  77.70449 -77.70449
## miss -40.49551  40.49551 -77.70449  77.70449
```

```
plotFit(fit, stat = "cov") # graphical test
```

Observed (red) and predicted (gray) covariances



Model Fit: Predicted vs. Observed Data

Testing model fit

- The statistics T1 and T2 quantify the discrepancy between observed and expected means/covariances (similar to Pearson's X^2)
- Posterior predictive p-values
 - Values around .50 indicate good model fit
 - Values close to 0 (or close to 1) indicate misfit
 - In contrast to frequentist p -values, PPP values are *not* uniformly distributed when generating data from the correct model

```
PPP(fit, M = 2000, nCPU = 4)
```

```
## ## Mean structure (T1):
## Observed = 0.0325755 ; Predicted = 0.03283945 ; p-value = 0.4885
##
## ## Covariance structure (T2):
## Observed = 6.487937 ; Predicted = 7.159747 ; p-value = 0.5245
##
## ## Individual fit (T1):
##      1      2      3      4      5      6      7      8      9     10     11     12     13
## 0.371 0.565 0.549 0.528 0.469 0.518 0.261 0.532 0.512 0.532 0.523 0.424 0.527
##     14     15     16     17     18     19     20     21     22     23     24     25     26
## 0.515 0.259 0.573 0.533 0.494 0.530 0.331 0.568 0.520 0.511 0.356 0.310 0.478
##     27     28     29     30     31     32     33     34     35     36     37     38     39
## 0.466 0.540 0.521 0.514 0.527 0.589 0.432 0.528 0.553 0.416 0.522 0.244 0.526
##     40     41     42     43     44     45     46     47     48     49     50
## 0.355 0.570 0.532 0.278 0.466 0.532 0.523 0.561 0.499 0.430 0.484
```

Modeling with TreeBUGS is simple

- 1 Define model and clean data
- 2 Draw MCMC samples
- 3 Check convergence
- 4 Check model fit
- 5 Interpret/plot parameters

Note that these are the usual steps in any Bayesian analysis. . .

Advanced Modeling

Assessing within-subject differences in parameters

- 1 Data: Additional columns for separate within-subject conditions
- 2 Model: Write EQN file for within-subject design
- 3 MCMC sampling (as usual)
- 4 Comparison: Compute differences of parameters (transformed parameters)

(1) Data structure for within-subject design

```
freq_within <- read.csv("2htm_within.csv")  
head(freq_within, 3)
```

##	high_cr	high_fa	high_hit	high_miss	low_cr	low_fa	low_hit	low_miss
## 1	45	5	46	4	35	15	44	6
## 2	40	10	42	8	36	14	31	19
## 3	48	2	46	4	36	14	35	15

Within-Subject Comparisons

(2) Model: Function for writing within-subject EQN files

- TreeBUGS has a function extends a standard MPT model to multiple within-subject conditions
- Essentially, model equations are copied and each parameter gets a new label (e.g., d_condition1)

```
# create EQN file for within-subject manipulations
withinSubjectEQN(htm_d,
                 labels = c("high", "low"), # factor labels
                 constant=c("g"))           # constant parameters
```

##	Tree	Category	Equation	EQN
## 1	high_target	high_hit	d_high	d
## 2	high_target	high_hit	$(1-d_{high}) * g$	$(1-d) * g$
## 3	high_target	high_miss	$(1-d_{high}) * (1-g)$	$(1-d) * (1-g)$
## 4	high_lure	high_cr	d_high	d
## 5	high_lure	high_fa	$(1-d_{high}) * g$	$(1-d) * g$
## 6	high_lure	high_cr	$(1-d_{high}) * (1-g)$	$(1-d) * (1-g)$
## 7	low_target	low_hit	d_low	d
## 8	low_target	low_hit	$(1-d_{low}) * g$	$(1-d) * g$
## 9	low_target	low_miss	$(1-d_{low}) * (1-g)$	$(1-d) * (1-g)$
## 10	low_lure	low_cr	d_low	d
## 11	low_lure	low_fa	$(1-d_{low}) * g$	$(1-d) * g$
## 12	low_lure	low_cr	$(1-d_{low}) * (1-g)$	$(1-d) * (1-g)$

(4) Transformed parameters

- Often the interest is in the difference of a parameter across conditions
 - Example: Difference in memory strength $\Delta_d = d_{\text{high}} - d_{\text{low}}$
- Based on the MCMC samples, we can simply compute any function of interest
 - We get a new set of posterior samples that can be summarized as usual

```
# fit to all conditions:
fit_within <- traitMPT("2htm_within.eqn", "2htm_within.csv")
```

```
# compute difference in d:
diff_d <- transformedParameters(
  fit_within,
  transformedParameters = list("diff_d = d_high - d_low"),
  level = "group")
summary(diff_d)$statistics
```

##	Mean	SD	Naive SE	Time-series SE
##	0.3217818803	0.0362304055	0.0003486272	0.0012263185

Comparisons in between-subject designs

- 1) Fit MPT model to each condition separately
 - Separate group-level parameters μ and Σ per group
- 2) Compute differences in group-level parameters across conditions/models

```
fit1 <- traitMPT(htm_d, "2htm.csv")
fit2 <- traitMPT(htm_d, "2htm_group2.csv")
```

```
diff_between <- betweenSubjectMPT(
  fit1, fit2,           # fitted MPT models
  par1 = "d",          # parameter to test
  stat = c("x-y", "x>y")) # transformed parameters
diff_between
```

	##	Mean	SD	2.5%	50%	97.5%	Time-series	SE	n.eff	Rhat	R_95%
## d.m1-d.m2	0.189	0.084	0.03	0.188	0.36		0.005	328	1.064	1.205	
## d.m1>d.m2	0.991	0.095	1.00	1.000	1.00		0.002	1505	1.124	1.162	

Regression of MPT parameters on covariates

- Example: Predict memory performance d as a function of age
- Statistically, this requires an regression extension to the model
- The latent probit values θ'_i are predicted by a design matrix X :

$$\theta'_i = \mu + X_i\beta + \delta_i$$

Implementation in TreeBUGS

- Requires only two new arguments to provide the data (age of persons) and the regression structure (predict parameter D_n by age)

```
fit_regression <- traitMPT(htm_d, data = "2htm.csv",  
                           covData = "covariates.csv",  
                           predStructure = list("d ; continuous"))  
  
round(fit_regression$summary$group$slope[,-6], 2)
```

##	Mean	SD	2.5%	50%	97.5%	n.eff	Rhat	R_95%
## slope_d_continuous	0.30	0.06	0.18	0.30	0.42	505	1.03	1.06
## slope_std_d_continuous	0.61	0.09	0.41	0.62	0.76	603	1.03	1.07

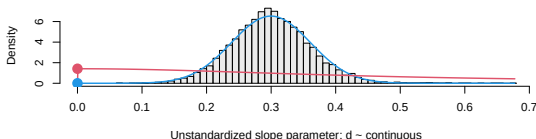
Bayes Factor for Covariate

Compute a Bayes factor

- H0: Slope parameter $\beta = 0$
- H1: Slope parameter $\beta \sim \text{Cauchy}(0, r)$ (with scale parameter r)
- Method: Savage-Dickey density ratio (Wagenmakers, 2010)
 - Bayes factor H1 vs. H0: prior divided by posterior density (at $\beta = 0$)
 - Only works for simple regression with 1 predictor (Heck, 2019)

```
BayesFactorSlope(fit_regression,  
                 parameter = "slope_d_continuous",  
                 direction = ">",           # H1: positive slope parameter  
                 plot = TRUE)              # plot Savage-Dickey density ratio
```

Bayes factor B_10=36362.895 (prior red; posterior blue)



```
##                               BF_0>    BF_>0  
## slope_d_continuous 2.750056e-05 36362.89
```

Between-Subject Comparisons: Similar to ANOVA

Between-subject designs: Assumptions about the covariance matrix Σ

- A) Separate covariance matrix per condition: $\Sigma_1, \Sigma_2, \dots$
 - See previous slides: `betweenSubjectMPT(fit1, fit2)`
- B) Identical covariance matrix Σ across conditions
 - Similar to ANOVA: “pooled variance” (Rouder & Morey; 2012)
 - Manipulation only affects the mean parameters μ

```
# fit all between-conditions jointly:
fit_between <- traitMPT(
  htm_d, "2htm.csv",
  covData = "covariates.csv",
  predStructure = list("d ; discrete"), # discrete predictor
  predType = c("c", "f")) # "c" =continuous; "f"=fixed-effects
```

```
# get estimates for the group-specific MPT parameters
gmeans <- getGroupMeans(fit_between)
round(gmeans, 2)
```

	Mean	SD	2.5%	50%	97.5%	p(one-sided vs. overall)
d_discrete[group_a]	0.47	0.09	0.30	0.47	0.65	0.02
d_discrete[group_b]	0.70	0.07	0.54	0.70	0.84	0.02

Sensitivity/robustness analysis

Define different priors

- Prior distributions in the latent-trait MPT necessary for:
 - Latent (probit-) mean μ
 - Latent (probit-) covariance matrix Σ : scaled inverse Wishart with
 - Prior matrix V
 - Degrees of freedom df
 - Scaling parameter ξ
- Example: We assume that guessing probabilities are around 50%

```
fit <- traitMPT(eqnfile="htm.txt", data="responses.csv",  
               restrictions=list("dn=do"),  
  
               mu = c(dn = "dnorm(0,1)",      # default prior  
                      g = "dnorm(0,5)"),      # prior focused around 50% guessing  
               xi = "dunif(0,2)",             # less dispersion of MPT parameters  
               V = diag(2),                   # default  
               df = 2 + 1)                    # default
```

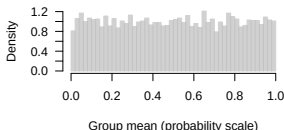
Understanding Priors

What do the priors actually mean?

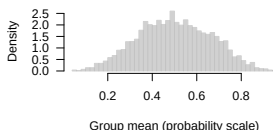
- Draw samples from the prior
- Plot mean/SD of MPT parameters

```
plotPrior(prior =  
  list(mu = c(dn = "dnorm(0,1)", # default prior  
             g = "dnorm(0,5)", # prior focused around 50%  
             xi="dunif(0,2)", # smaller scale for group SD  
             V= diag(2), df = 3)) # default Wishart prior
```

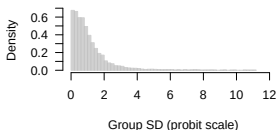
Prior on group mean: $\mu = \text{dnorm}(0,1)$



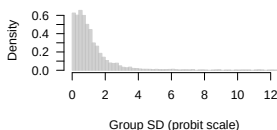
Prior on group mean: $\mu = \text{dnorm}(0,5)$



Prior on group SD: $\xi = \text{dunif}(0,2)$



Prior on group SD: $\xi = \text{dunif}(0,2)$

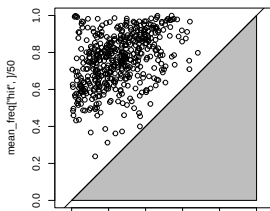


Prior Predictive Sampling

Prior predictive distribution

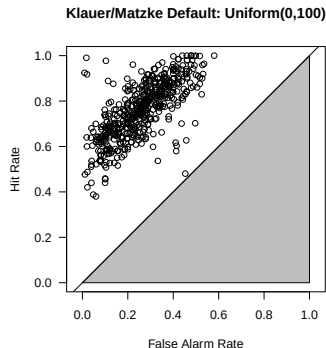
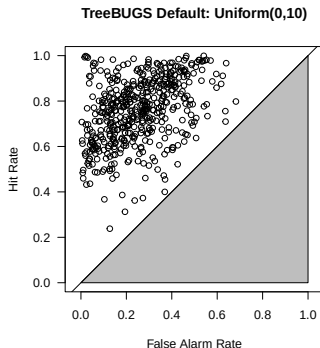
- 1 Draw samples from the prior
- 2 Draw new data (response frequencies)
- 3 Assess predicted data (e.g., plots or descriptive statistics)

```
pp <- priorPredictive(prior = list(mu = "dnorm(0,1)", xi="dunif(0,10)",  
                                V=diag(2), df=2+1),  
                    eqnfile = htm, restrictions = list("dn=do"),  
                    numItems = c(target = 50, lure = 50),  
                    N = 50, M = 500)      # number of participants/samples  
  
# compute and plot predicted values for the average hit/FA rates (in ROC space)  
mean_freq <- sapply(pp, colMeans)  
par(mar=c(4,5,.1, .1))  
plot(mean_freq["fa",]/50, mean_freq["hit",]/50, asp = 1, xlim =0:1, ylim=0:1)  
polygon(c(0,1,1), c(0,0,1), col = "gray")  
abline(0, 1)
```



Excursion: Different default priors for the scale parameter ξ

- 1 TreeBUGS (Heck et al., 2018): $\xi \sim \text{Uniform}(0, 10)$
- 2 Klauer (2010) and Matzke et al. (2015): $\xi \sim \text{Uniform}(0, 100)$



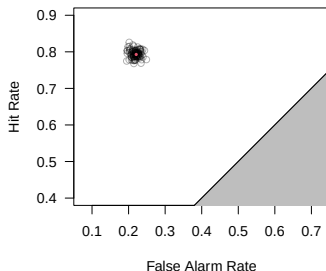
Posterior Predictive

Posterior Predictive Distribution

- What data does the fitted model predict?
- Use posterior samples of the *parameters* to draw new samples of the *data* (i.e., predicted response frequencies)
 - Note: These are the basis of posterior-predictive checks (T1 and T2 statistics)

```
postpred <- posteriorPredictive(fit, M = 100, nCPU = 4)
```

Posterior predicted (mean frequency)



Sensitivity and robustness analysis

- Assessing the impact of priors, estimate necessary sample size for specific analysis etc.
 - 1 Generate data from (correct or wrong) model
 - 2 Fit model
 - 3 If necessary: Replicate with a for-loop

```
# standard, fixed-effects MPT (generate data for one person)
sim <- genMPT(theta = c(d = .7, g = .5),
              numItems = c(target = 50, lure = 50),
              eqnfile = htm_d)

# hierarchical MPT (generate a complete table of frequencies)
sim2 <- genTraitMPT(N = 100, eqnfile = htm_d,
                   mean = c(d = .7, g = .5),
                   sigma = c(d = .4, g = .2),
                   rho = diag(2))
```

Appendix

Appendix: Testing for Heterogeneity

Test by Smith and Batchelder (2008)

- A) Test person heterogeneity assuming items homogeneity (χ^2)
- B) Test person heterogeneity under item heterogeneity (permutation bootstrap)

```
# A) chi^2 test
test <- testHetChi(freq = frequencies,
                  tree = c(cr="lure", fa="lure",
                          hit="target", miss="target"))
data.frame(test)
```

```
##          chisq df          prob
## 1 851.3646 98 1.916631e-120
```

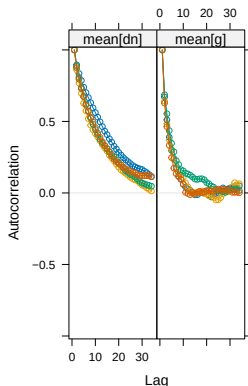
```
# B) requires data in long format (variables: person / item / response)
# testHetPerm(data, tree, source = "person")
```

Appendix: Convergence

Autocorrelation function

- How strongly are the MCMC samples correlated between iteration t and iteration $t + \text{Lag}$?
- Ideally, these curves should rapidly decrease towards zero.

```
plot(fit, parameter = "mean", type = "acf")
```

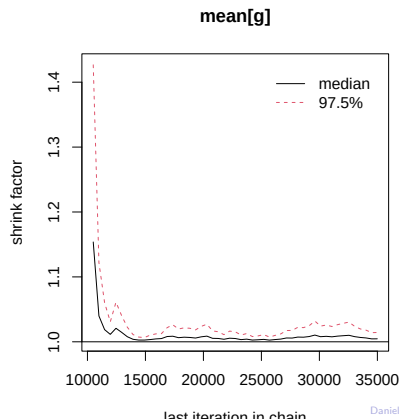
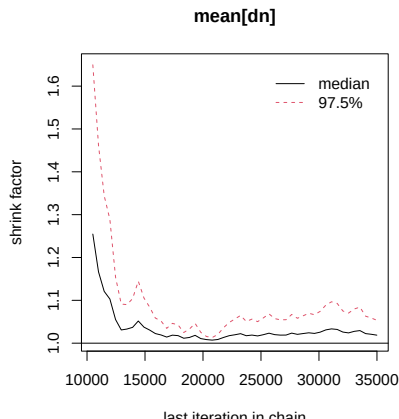


Appendix: Convergence

Plot the evolution of the Gelman-Rubin statistic

- Also known as: “potential scale reduction factor” or “R hat”
- Similar to ANOVA: Compares between-chain and within-chain variances (large differences between these variances indicate nonconvergence)
- Statistic should be close to 1

```
plot(fit, parameter = "mean", type = "gelman")
```



MCMC samplers available in TreeBUGS

- Fixed-effects (“standard”) MPT: Gibbs sampler in C++
- Beta-MPT: JAGS and C++
- Latent-trait with extensions: JAGS
 - Combination of random- and fixed-effects parameters
 - Continuous and categorical covariates
 - Group-level structure: Independent normal distributions

References I

- Heck, D. W. (2019). A caveat on the Savage-Dickey density ratio: The case of computing Bayes factors for regression parameters. *British Journal of Mathematical and Statistical Psychology*, 72:316–333.
- Heck, D. W., Arnold, N. R., and Arnold, D. (2018). TreeBUGS: An R package for hierarchical multinomial-processing-tree modeling. *Behavior Research Methods*, 50:264–284.
- Klauer, K. C. (2010). Hierarchical multinomial processing tree models: A latent-trait approach. *Psychometrika*, 75:70–98.
- Matzke, D., Dolan, C. V., Batchelder, W. H., and Wagenmakers, E.-J. (2015). Bayesian estimation of multinomial processing tree models with heterogeneity in participants and items. *Psychometrika*, 80:205–235.
- Michalkiewicz, M. and Erdfelder, E. (2016). Individual differences in use of the recognition heuristic are stable across time, choice objects, domains, and presentation formats. *Memory & Cognition*, 44:454–468.
- Plummer, M. (2003). JAGS: A program for analysis of Bayesian graphical models using Gibbs sampling. In *Proceedings of the 3rd International Workshop on Distributed Statistical Computing*, volume 124, page 125. Vienna, Austria.

Smith, J. B. and Batchelder, W. H. (2008). Assessing individual differences in categorical data. *Psychonomic Bulletin & Review*, 15:713–731.